

# main

August 1, 2026

```
[1]: num1 = 10_00_000  
      print(num1)
```

1000000

```
[2]: print(format(0.1, ".25f"))
```

0.10000000000000000055511151

```
[3]: print(format(0.125, "0.25f"))
```

0.12500000000000000000000000

```
[4]: print(0.1 + 0.1 + 0.1 == 0.3)
```

False

```
[5]: print(format(0.1, ".25f"))
```

0.10000000000000000055511151

```
[6]: print(format(0.3, ".25f"))
```

0.2999999999999999888977698

```
[7]: print(format(0.2, ".25f"))
```

0.2000000000000000111022302

```
[8]: print(abs((0.1 + 0.1 + 0.1) - 0.3) < 0.001)
```

True

```
[9]: print(0.125.as_integer_ratio())
```

(1, 8)

```
[10]: print((10).__add__(25))
```

35

```
[11]: a = float(10)  
      print(a)
```

10.0

```
[12]: c = (-4) ** 0.5  
      print(c)
```

(1.2246467991473532e-16+2j)

```
[13]: print(type(c))
```

<class 'complex'>

```
[14]: print(c.real)
```

1.2246467991473532e-16

```
[15]: print(c.imag)
```

2.0

```
[16]: c = 10 + 2j  
      print(type(c))
```

<class 'complex'>

```
[17]: print(c)
```

(10+2j)

```
[18]: try:  
      print("2" < 2)  
      except TypeError:  
          pass
```

```
[19]: print("2" == 2)
```

False

```
[20]: a = 0  
      b = 1  
      c = 500  
      d = 0  
      e = 1  
      f = 500  
      print(id(a), id(b), id(c), id(d), id(e), id(f))
```

11370000 11370032 132956150733552 11370000 11370032 132956150733904

```
[21]: print(0.1 * 3 - 0.3 < abs(1e-6))
```

True

```
[22]: a = 257
      print(a is 257)
```

False

```
<>:2: SyntaxWarning: "is" with 'int' literal. Did you mean "=="?
<>:2: SyntaxWarning: "is" with 'int' literal. Did you mean "=="?
/tmp/ipykernel_2199205/776835706.py:2: SyntaxWarning: "is" with 'int' literal.
Did you mean "=="?
      print(a is 257)
```

```
[23]: a = 10
      b = 0
      print(b != 0 and a / b)
```

False

```
[24]: a = 1e-6
      b = 2e-6
      tol = 1e-7
      print(abs(a - b) <= tol)
```

False

```
[25]: elapsed = 7835
      hours = elapsed // 3600
      minutes = (elapsed - (hours * 3600)) // 60
      seconds = elapsed - ((hours * 3600) + (minutes * 60))
      print(hours, minutes, seconds)
```

2 10 35

```
[26]: elapsed = 7835
      hours = elapsed // (60 * 60)
      minutes = (elapsed % (60 * 60)) // 60
      seconds = (elapsed % (60 * 60)) % 60
      print(hours, minutes, seconds)
```

2 10 35

```
[27]: a = None
      if a == None:
          a = "N/A"
      print(a)
```

N/A

```
[28]: a = None
      a = "N/A" if a is None else a
      print(a)
```

N/A

```
[29]: score = 700
rating = "Invalid"
if score < 0:
    rating = "Negative"
elif score < 580:
    rating = "Poor"
elif score < 670:
    rating = "Fair"
elif score < 740:
    rating = "Good"
elif score < 800:
    rating = "Very Good"
elif score < 850:
    rating = "Excellent"
print(rating)
```

Good

```
[30]: elapsed = 1000000
magnitude = "Invalid"
if elapsed < 60:
    magnitude = "seconds"
elif elapsed < (60 * 60):
    magnitude = "minutes"
elif elapsed < (60 * 60 * 24):
    magnitude = "hours"
elif elapsed < (60 * 60 * 24 * 7):
    magnitude = "days"
elif elapsed > (60 * 60 * 24 * 7):
    magnitude = "weeks"
print(magnitude)
```

weeks

```
[31]: t = [1, 2], 3, 4
l = list(t)
print(l)
```

[[1, 2], 3, 4]

```
[32]: l[0][1] = 20
print(l)
```

[[1, 20], 3, 4]

```
[33]: print(t)
```

([1, 20], 3, 4)

```
[34]: for i in t:  
        print(id(i))
```

132956148962240  
11370096  
11370128

```
[35]: for i in l:  
        print(id(i))
```

132956148962240  
11370096  
11370128

```
[36]: for i, j in zip(l, t):  
        print(i is j)
```

True  
True  
True

```
[37]: s = "ABCDEFGHIJ"  
        print(s[3:6:-1])
```

```
[38]: print(s[3:5 + 1])
```

DEF

```
[39]: m1 = [[0, 0, 0], [0, 0, 0], [0, 0, 0]]  
        m2 = m1[0:2]  
        print(m1[0] is m2[1])
```

False

```
[40]: m1[0] = "Python"  
        print(m1)
```

['Python', [0, 0, 0], [0, 0, 0]]

```
[41]: print(m2)
```

[[0, 0, 0], [0, 0, 0]]

```
[42]: m1[1][0] = 100
```

```
[43]: print(m1)
```

['Python', [100, 0, 0], [0, 0, 0]]

```
[44]: print(m2)
[[0, 0, 0], [100, 0, 0]]

[45]: print("abcdef"[-1:-4])

[46]: print("abcdef"[1:4:-1])

[47]: print("racecar" == "racecar"[::-1])
True

[48]: list1 = [0, 1, 2, 3, 4]
list1[0:3] = "abc"
print(list1)
['a', 'b', 'c', 3, 4]

[49]: list1[0:3] = ["a", "b", "c"]
print(list1)
['a', 'b', 'c', 3, 4]

[50]: list1[0:3] = ["a", "b"]
print(list1)
['a', 'b', 3, 4]

[51]: del list1[2]
print(list1)
['a', 'b', 4]

[52]: del list1[0:2]
print(list1)
[4]

[53]: list1.extend("ABC")
print(list1)
[4, 'A', 'B', 'C']

[54]: list1.insert(0, "YO!")
print(list1)
['YO!', 4, 'A', 'B', 'C']
```

```
[55]: list1[:-3:-1] = "cb"  
print(list1)
```

```
['Y0!', 4, 'A', 'b', 'c']
```

```
[ ]: import timeit
```

```
l = []  
print(timeit.timeit("l.append(1)", globals = globals(), number = 100_000))
```

```
0.0022154279868118465
```

```
[57]: print(len(l))
```

```
100000
```

```
[58]: list2 = []  
print(timeit.timeit("list2.insert(0, 1)", globals = globals(), number = 100_000))
```

```
1.5951547129661776
```

```
[59]: print(len(list2))
```

```
100000
```

```
[60]: print(1.249788 / 0.0018043)
```

```
692.6719503408524
```

```
[61]: list2 = []  
print(timeit.timeit("list2.insert(len(list2), 1)", globals = globals(), number = 100_000))
```

```
0.0049534509889781475
```

```
[62]: print(0.0057634 / 0.0018043)
```

```
3.194258161059691
```

```
[63]: l1 = [[0, 1], 2, "3"]  
l2 = l1[:]  
for i in l1:  
    print(id(i))
```

```
132956148995904
```

```
11370064
```

```
11434512
```

```
[64]: for i in l2:  
    print(id(i))
```

132956148995904  
11370064  
11434512

```
[65]: 12[0][0] = "0"  
      for i in l1:  
          print(id(i))
```

132956148995904  
11370064  
11434512

```
[66]: for i in l2:  
      print(id(i))
```

132956148995904  
11370064  
11434512

```
[67]: for i in l1:  
      print(i)
```

['0', 1]  
2  
3

```
[68]: for i in l2:  
      print(i)
```

['0', 1]  
2  
3

```
[69]: 12[1] = "22"  
      for i in l1:  
          print(id(i))
```

132956148995904  
11370064  
11434512

```
[70]: for i in l2:  
      print(id(i))
```

132956148995904  
132956222660720  
11434512

```
[71]: for i in l1:  
      print(i)
```

```
['0', 1]
2
3
```

```
[72]: for i in l2:
        print(i)
```

```
['0', 1]
22
3
```

```
[73]: l1 = [[0, 1], 2, "3"]
      l2 = l1[:]
      l2[0] = ["0", 1]
      for i in l1:
          print(id(i))
```

```
132956149024704
11370064
11434512
```

```
[74]: for i in l2:
        print(id(i))
```

```
132956149024512
11370064
11434512
```

```
[75]: for i in l1:
        print(i)
```

```
[0, 1]
2
3
```

```
[76]: for i in l2:
        print(i)
```

```
['0', 1]
2
3
```

```
[77]: l1 = [[0, 1], 2, "3"]
      l2 = l1[:]
      print(id(l1), id(l2))
```

```
132956149024768 132956149024448
```

```
[78]: for i, j in zip(l1, l2):
        print(id(i), id(j))
```

```
132956149024384 132956149024384
11370064 11370064
11434512 11434512
```

```
[79]: for i, j in zip(l1[0], l2[0]):
        print(id(i), id(j))
```

```
11370000 11370000
11370032 11370032
```

```
[80]: l1 = [[0, 1], 2, "3"]
        l2 = l1.copy()
        print(id(l1), id(l2))
```

```
132956149023232 132956149022784
```

```
[81]: for i, j in zip(l1, l2):
        print(id(i), id(j))
```

```
132956148831040 132956148831040
11370064 11370064
11434512 11434512
```

```
[82]: for i, j in zip(l1[0], l2[0]):
        print(id(i), id(j))
```

```
11370000 11370000
11370032 11370032
```

```
[ ]: import copy
```

```
l1 = [[0, 1], 2, "3"]
l2 = copy.deepcopy(l1)
print(id(l1), id(l2))
```

```
132956149028096 132956149027328
```

```
[84]: for i, j in zip(l1, l2):
        print(id(i), id(j))
```

```
132956149028480 132956148754752
11370064 11370064
11434512 11434512
```

```
[85]: for i, j in zip(l1[0], l2[0]):
        print(id(i), id(j))
```

```
11370000 11370000
11370032 11370032
```

```
[86]: t1 = (0, "1", "23")
      t2 = t1[:]
      print(t1 is t2)
```

True

```
[87]: a, b, c, d = "abcd"
      print(a, b, c, d)
```

a b c d

```
[88]: s = 'FfEeDdCcBbAa'
      lowerLetters = s[::-2]
      upperLetters = s[-2::-2]
      print(lowerLetters)
```

abcdef

```
[89]: print(upperLetters)
```

ABCDEF

```
[90]: t1 = 1, 2, 3, 4, 5, 6
      t2 = 7, 8, 9, 10
      t3 = 11, 12, 13, 14, 15, 16, 17
      t = t1 + t2 + t3
      l = list(t)
      l[::2] = [0] * len(l[::2])
      print(tuple(l))
```

(0, 2, 0, 4, 0, 6, 0, 8, 0, 10, 0, 12, 0, 14, 0, 16, 0)

```
[91]: m = [[0, 0, 0], [0, 0, 0], [0, 0, 0]]
      m[0][0] = 1
      m[1][1] = 1
      m[2][2] = 1
      print(m)
```

[[1, 0, 0], [0, 1, 0], [0, 0, 1]]

```
[92]: m = [[0, 0, 0], [0, 0, 0], [0, 0, 0]]
      m1 = copy.deepcopy(m)
      m1[0][0] = 1
      m1[1][1] = 1
      m1[2][2] = 1
      print(m1)
```

[[1, 0, 0], [0, 1, 0], [0, 0, 1]]

```
[93]: print(m)
```

[[0, 0, 0], [0, 0, 0], [0, 0, 0]]

```
[94]: data = [(100, 'USD', 'EUR', 0.83), (100, 'USD', 'CAD', 1.27), (100, 'CAD', 'EUR', 0.65)]
      USDAmountInEUR = data[0][0] * data[0][3]
      print(str(data[0][0]) + " " + data[0][1] + " = " + str(USDAmountInEUR) + " " + str(data[0][2]))
```

100 USD = 83.0 EUR

```
[95]: USDAmountInCAD = data[1][0] * data[1][3]
      print(str(data[1][0]) + " " + data[1][1] + " = " + str(USDAmountInCAD) + " " + str(data[1][2]))
```

100 USD = 127.0 CAD

```
[96]: CADAmountInEUR = data[2][0] * data[2][3]
      print(str(data[2][0]) + " " + data[2][1] + " = " + str(CADAmountInEUR) + " " + str(data[2][2]))
```

100 CAD = 65.0 EUR

```
[97]: print(ord("A"))
```

65

```
[98]: print("\N{snake}")
```

```
[99]: print(ord("\N{snake}"))
```

128013

```
[100]: print(hex(ord("\N{snake}")))
```

0x1f40d

```
[101]: print("\u03b1")
```

```
[102]: print("\U0001F40D")
```

```
[103]: print(int("1f40d", 16))
```

128013

```
[104]: print("\N{SNAKE} is a snake")
```

is a snake

```
[105]: print(int("7ff", 16))
```

2047

```
[106]: print(int("ffff", 16))
```

65535

```
[107]: print(int("10ffff", 16))
```

1114111

```
[108]: print(bin(2047))
```

0b1111111111

```
[109]: print(len(bin(2047)))
```

13

```
[110]: print(bin(65535))
```

0b1111111111111111

```
[111]: print(len(bin(65535)))
```

18

```
[112]: print(bin(1114111))
```

0b10000111111111111111

```
[113]: print(len(bin(1114111)))
```

23

```
[114]: print(bin(128013))
```

0b11111010000001101

```
[115]: print("\n{snake}")
```

{snake}

```
[116]: a = True
      b = True
      print(a is b)
```

True

```
[117]: a = False
      b = False
      print(a is b)
```

True

```
[118]: a = True  
       print(id(a))
```

11120192

```
[119]: a = False  
       print(id(a))
```

11045056

```
[120]: b = False  
       print(id(b))
```

11045056

```
[121]: ?str.find
```

Docstring:

Return the lowest index in S where substring sub is found, such that sub is contained within S[start:end].

Optional arguments start and end are interpreted as in slice notation.

Return -1 on failure.

Type: method\_descriptor

```
[122]: print(f"1 + 1 = {1 + 1}")
```

1 + 1 = 2

```
[123]: bid = 1.5760  
       ask = 1.5763  
       print("bid: {}, ask: {}, spread: {}".format(bid, ask, ask - bid))
```

bid: 1.576, ask: 1.5763, spread: 0.00029999999999999696

```
[124]: print("bid: {:.4f}, ask: {:.4f}, spread: {:.4f}".format(bid, ask, ask - bid))
```

bid: 1.5760, ask: 1.5763, spread: 0.0003

```
[125]: print(f"bid: {bid:.4f}, ask: {ask:.4f}, spread: {ask - bid:.4f}")
```

bid: 1.5760, ask: 1.5763, spread: 0.0003

```
[126]: print(f"bid: {bid:.4f}, ask: {ask:.4f}, spread: {(ask - bid):.4f}")
```

bid: 1.5760, ask: 1.5763, spread: 0.0003

```
[127]: string1 = ", ß, "  
       string1 = 'Π, , , , '
```

```
chars = string1.split(", ")
print(chars)
```

```
['Π', ' ', ' ', ' ', ' ', ' ']
```

```
[128]: list1 = []
for c in chars:
    list1.append(hex(ord(c)))
print(list1)
```

```
['0x3a0', '0x3cd', '0x3b8', '0x3c9', '0x3bd']
```

```
[129]: list1 = []
for c in chars:
    list1.append(c.upper())
print(list1)
```

```
['Π', 'T', 'Θ', 'Ω', 'N']
```

```
[130]: list1 = []
for c in chars:
    list1.append(c.lower())
print(list1)
```

```
[' ', ' ', ' ', ' ', ' ']
```

```
[131]: a = 42
print(f"The number {a} is not even") if a % 2 else print(f"The number {a} is_
↪even")
```

The number 42 is even

```
[132]: a = 41
print(f"The number {a} is not even") if a % 2 else print(f"The number {a} is_
↪even")
```

The number 41 is not even

```
[133]: a = 42
print("The number {} is not even".format(a)) if a % 2 else print("The number {}_
↪is even".format(a))
```

The number 42 is even

```
[134]: a = 41
print("The number {} is not even".format(a)) if a % 2 else print("The number {}_
↪is even".format(a))
```

The number 41 is not even

```
[135]: a = 0.3
      b = 0.1
      print(f"{a} / {b} = {a / b:.4f}")
```

0.3 / 0.1 = 3.0000

```
[136]: a = 42
      qualifier = "is not" if a % 2 else "is"
      print(f"The number {a} {qualifier} even")
```

The number 42 is even

```
[137]: a = 421
      qualifier = "is not" if a % 2 else "is"
      print(f"The number {a} {qualifier} even")
```

The number 421 is not even

```
[138]: print(range(10))
```

range(0, 10)

```
[139]: print(type(range(10)))
```

<class 'range'>

```
[140]: for i in range(3):
      print(i)
      print(i)
```

0  
1  
2  
2

```
[141]: for i in "abc":
      pass
      print(i)
```

c

```
[142]: raggedArray = [[1, 2], [3, 4, 5, 6], [7, 8, 9], [10]]
      for r in range(len(raggedArray)):
          for c in range(len(raggedArray[r])):
              print(f"[{r}, {c}] = {raggedArray[r][c]}")
      print("-" * 5)
```

[0, 0] = 1  
[0, 1] = 2  
-----  
[1, 0] = 3  
[1, 1] = 4

```

[1, 2] = 5
[1, 3] = 6
-----
[2, 0] = 7
[2, 1] = 8
[2, 2] = 9
-----
[3, 0] = 10
-----

```

```

[143]: identityMatrix = []
size = 10
for r in range(size):
    row = []
    for c in range(size):
        if r == c:
            row.append(1)
        else:
            row.append(0)
    identityMatrix.append(row)
identityMatrix

```

```

[143]: [[1, 0, 0, 0, 0, 0, 0, 0, 0, 0],
        [0, 1, 0, 0, 0, 0, 0, 0, 0, 0],
        [0, 0, 1, 0, 0, 0, 0, 0, 0, 0],
        [0, 0, 0, 1, 0, 0, 0, 0, 0, 0],
        [0, 0, 0, 0, 1, 0, 0, 0, 0, 0],
        [0, 0, 0, 0, 0, 1, 0, 0, 0, 0],
        [0, 0, 0, 0, 0, 0, 1, 0, 0, 0],
        [0, 0, 0, 0, 0, 0, 0, 1, 0, 0],
        [0, 0, 0, 0, 0, 0, 0, 0, 1, 0],
        [0, 0, 0, 0, 0, 0, 0, 0, 0, 1]]

```

```

[144]: raggedArray = [[1, 2], [3, 4, 5, 6], [7, 8, 9], [10]]
for i, r in enumerate(raggedArray):
    for j, e in enumerate(raggedArray[i]): # e for element, cuz it's not a
        ↪column technically
        print(f"[{i}, {j}] = {raggedArray[i][j]}")
    print("-" * 8)

```

```

[0, 0] = 1
[0, 1] = 2
-----
[1, 0] = 3
[1, 1] = 4
[1, 2] = 5
[1, 3] = 6
-----

```

```
[2, 0] = 7
[2, 1] = 8
[2, 2] = 9
-----
[3, 0] = 10
-----
```

```
[145]: print(f"{0.1:.4f}")
```

```
0.1000
```

```
[146]: print("{:.4f}".format(0.1))
```

```
0.1000
```

```
[147]: total, count = (0,) * 2
data = [10.5, 11.2, 9.8, None, 11.5, None]
for e in data:
    if e:
        total += e
        count += 1
average = total / count
print(average)
```

```
10.75
```

```
[148]: for i, e in enumerate(data):
        if not e:
            data[i] = average
print(data)
```

```
[10.5, 11.2, 9.8, 10.75, 11.5, 10.75]
```

```
[149]: # data = [10.5, 11.2, 9.8, None, 11.5, None]
# count = sum(1 for e in data if e)
# total = sum(e for e in data if e)
# average = total / count
# print(sum, average)
```

```
[ ]: import statistics
```

```
data = [10.5, 11.2, 9.8, None, 11.5, None]
average = statistics.fmean(e for e in data if e)
data = [average if e is None else e for e in data]
print(data)
```

```
[10.5, 11.2, 9.8, 10.75, 11.5, 10.75]
```

```
[ ]: import pandas

data = [10.5, 11.2, 9.8, None, 11.5, None]
df = pandas.DataFrame(data)
df.fillna(df.mean(), inplace = True)
print(df[0].tolist())
```

```
[10.5, 11.2, 9.8, 10.75, 11.5, 10.75]
```

```
[152]: print(len(str((1, 2))))
```

```
6
```

```
[153]: print(str((1, 2)))
```

```
(1, 2)
```